Vigil MDR Setup Guide

Build Vigil from nothing on a Windows laptop. No earlier knowledge of the project is needed. Allow about half a day.

What you'll build

A Windows test machine sends security events to LimaCharlie. Detection rules there raise alerts. A background job on your laptop downloads those alerts, groups them into incidents, asks a local AI model for an opinion, and saves everything in PostgreSQL. You review incidents in a web portal.

> **Before you start:** Vigil is a learning lab. It doesn't stop attacks, and its AI verdicts can be wrong, so a person must review them. Only run attack simulations on a test machine you own or are allowed to use, never on your laptop or a work machine.

What you need

Item	Details
Windows 10 or 11 laptop	16 GB of RAM recommended, plus about 10 GB of free disk space
Test machine	A Windows virtual machine, using VirtualBox or similar
Accounts	Free accounts at LimaCharlie and GitHub, with access to the Vigil repo
Tools	Python 3.11 or later, Git, Ollama, PostgreSQL 17 and cloudflared. Step 2 installs them

Run every laptop command in **PowerShell**. Open a new window whenever a step tells you to, because new settings only reach new windows.

Step 1: Set up LimaCharlie and the test machine

1. Create a virtual machine, then install Windows on it.
2. Set its network to NAT, and turn off shared folders and the shared clipboard.
3. Take a snapshot named `clean-base`.
4. Create an organization at app.limacharlie.io.
5. Create an installation key, then follow LimaCharlie's Windows sensor instructions on the virtual machine.
6. Enable **Insight**, which stores detection history.
7. Add detection rules. Enable the Sigma and managed rule sets LimaCharlie offers under Extensions or Add-ons.
8. Open **Access Management**, then **REST API**, and copy your Organization ID.

9. Create an API key named `vigil-poller` with only the `insight.det.get` permission. Copy it somewhere safe, because it may only be shown once.

> Don't use the Org JWT shown on that page. It's a temporary token with wide access. Menu names may differ from these labels.

> **Check:** run `whoami /all` on the virtual machine, then find it in the sensor's timeline in LimaCharlie.

Step 2: Install the tools on your laptop

```
winget install --id Python.Python.3.12 --exact
winget install --id Git.Git --exact
winget install --id GitHub.cli --exact
winget install --id Ollama.Ollama --exact
winget install --id Cloudflare.cloudflared --exact
winget install --id PostgreSQL.PostgreSQL.17 --exact
```

PostgreSQL's installer asks you to choose a password for its `postgres` admin account. Keep it for Step 5. Close PowerShell and open a new window, then check the tools:

```
python --version
git --version
ollama --version
cloudflared --version
```

If PostgreSQL won't download

If winget reports a 403 error, install PostgreSQL from conda-forge. This needs no admin rights. Change `F:\PostgreSQL` if you want it elsewhere.

```
$root = 'F:\PostgreSQL'
New-Item -ItemType Directory -Force $root | Out-Null
curl.exe -L -o "$root\micromamba.exe" 'https://github.com/mamba-org/micromamba-
releases/releases/latest/download/micromamba-win-64.exe'
$env:MAMBA_ROOT_PREFIX = "$root\mamba"
& "$root\micromamba.exe" create -y -p "$root\env" -c conda-forge --override-channels
'postgresql=17'
$pwFile = "$env:TEMP\pg-admin.txt"
$adminPassword = Read-Host 'Choose a postgres admin password'
[IO.File]::WriteAllText($pwFile, $adminPassword)
& "$root\env\Library\bin\initdb.exe" -D "$root\data" -U postgres --pwfile=$pwFile -A
scram-sha-256 -E UTF8 --locale=C
Remove-Item -LiteralPath $pwFile
Add-Content -LiteralPath "$root\data\postgresql.conf" -Value "`nlisten_addresses =
'localhost'`nport = 5432" -Encoding ascii
& "$root\env\Library\bin\pg_ctl.exe" start -D "$root\data" -l "$root\postgres.log"
```

Step 3: Get the code

This repo is private, so your GitHub account needs access. Then run:

```
gh auth login
cd $HOME
git clone https://github.com/sunilnaidu265/vigil-mdr.git
cd vigil-mdr
python -m venv .venv
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
New-Item -ItemType Directory -Force data | Out-Null
```

> **Check:** your prompt starts with `(.venv)`.

Step 4: Set up Ollama

```
ollama pull qwen3:4b
ollama list
```

> **Check:** `qwen3:4b` appears in the list.

Step 5: Create the database

Choose the line that matches how you installed PostgreSQL, then run the rest.

```
# winget install
$bin = 'C:\Program Files\PostgreSQL\17\bin'
# conda-forge install
$bin = 'F:\PostgreSQL\env\Library\bin'

$bytes = New-Object byte[] 24
[Security.Cryptography.RandomNumberGenerator]::Create().GetBytes($bytes)
$dbPassword = [BitConverter]::ToString($bytes).Replace('-', '')
[Environment]::SetEnvironmentVariable('VIGIL_DB_PASSWORD', $dbPassword, 'User')
$env:VIGIL_DB_PASSWORD = $dbPassword
$env:PGPASSWORD = Read-Host 'Enter the postgres admin password'
"CREATE ROLE vigil LOGIN PASSWORD '$dbPassword';`nCREATE DATABASE vigil OWNER vigil;" |
& "$bin\psql.exe" -h localhost -U postgres -d postgres -q -v ON_ERROR_STOP=1
Remove-Item Env:PGPASSWORD
.\.venv\Scripts\python.exe db.py
```

> **Check:** the last command prints `tables ready`.

Step 6: Save your LimaCharlie details

Replace both placeholders with your values. Never put them in code or commit them to Git.

```
[Environment]::SetEnvironmentVariable('LC_OID', 'your-org-id', 'User')
[Environment]::SetEnvironmentVariable('LC_API_KEY', 'your-api-key', 'User')
```

Close PowerShell and open a new window. Go back to the project folder, then check:

```
cd $HOME\vigil-mdr
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
[bool]$env:LC_OID; [bool]$env:LC_API_KEY; [bool]$env:VIGIL_DB_PASSWORD
```

> **Check:** all three lines print `True`.

Step 7: Create a test detection

Run this **on the virtual machine only**. It's a harmless encoded command that prints text.

```
powershell.exe -nop -enc VwByAGkAdABlAC0ASABvAHMAdAAgAFYAaQBnAGkAbAAgAHQAZQBzAHQA
```

For more realistic data, install Atomic Red Team on the virtual machine, and run a test:

```
IEX (IWR 'https://raw.githubusercontent.com/redcanaryco/invoke-
atomicredteam/master/install-atomicredteam.ps1' -UseBasicParsing)
Install-AtomicRedTeam -getAtomics
Invoke-AtomicTest T1059.001 -ShowDetailsBrief
Invoke-AtomicTest T1059.001 -TestNumbers 11
```

> Read what a test does with `-ShowDetailsBrief` before running it. Test numbers can change. Restore your snapshot when you've finished testing.

> **Check:** your rule's detection appears on LimaCharlie's Detections page.

Step 8: Run the pipeline once

Back on your laptop, run this in the project folder. The first run looks back 3 days and can take several minutes.

```
.\.venv\Scripts\python.exe poll.py
Get-Content data\poll.log -Tail 10
```

> **Check:** the log shows a line starting `fetched`, with no `run failed`. Incidents wait 10 minutes after their last detection, so run it again later if none are analyzed yet.

Step 9: Run it automatically

Run this in the project folder. It starts the job every 5 minutes with no window.

```
$dir = (Get-Location).Path
$action = New-ScheduledTaskAction -Execute "$dir\.venv\Scripts\pythonw.exe" -Argument
"`"$dir\poll.py`"" -WorkingDirectory $dir
$trigger = New-ScheduledTaskTrigger -Once -At (Get-Date).AddMinutes(1) -
RepetitionInterval (New-TimeSpan -Minutes 5) -RepetitionDuration (New-TimeSpan -Days
3650)
$settings = New-ScheduledTaskSettingsSet -StartWhenAvailable -AllowStartIfOnBatteries -
DontStopIfGoingOnBatteries -MultipleInstances IgnoreNew -ExecutionTimeLimit (New-
TimeSpan -Hours 1)
Register-ScheduledTask -TaskName 'Vigil Poller' -Action $action -Trigger $trigger -
Settings $settings
```

If you used the conda-forge install, also start PostgreSQL at login:

```
$cmd = "& 'F:\PostgreSQL\env\Library\bin\pg_ctl.exe' start -D 'F:\PostgreSQL\data' -l
'F:\PostgreSQL\postgres.log'"
$action = New-ScheduledTaskAction -Execute 'powershell.exe' -Argument "-NoProfile -
WindowStyle Hidden -Command `"$cmd`""
$trigger = New-ScheduledTaskTrigger -AtLogOn -User "$env:USERDOMAIN\$env:USERNAME"
Register-ScheduledTask -TaskName 'Vigil PostgreSQL' -Action $action -Trigger $trigger
```

> **Check:** after 5 minutes, this shows a recent `LastRunTime` and a `LastTaskResult` of `0`.

```
Get-ScheduledTaskInfo -TaskName 'Vigil Poller' | Select-Object LastRunTime,
LastTaskResult, NextRunTime
```

Step 10: Tell Vigil about your own tools

Add any trusted automation or management tools to `KNOWN_TOOLS` in `correlate.py`, using the lowercase full path. This lowers their score and tells the AI what they are. Remove the sample Orborus entry if you don't use it.

```
KNOWN_TOOLS = {r"c:\program files\yourtool\agent.exe": "Your management agent"}
```

Step 11: Start the web portal

Choose a portal password of at least 14 characters. Nothing appears while you type it.

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File tools\set-portal-password.ps1
```

Open a new PowerShell window, then start the portal and leave the window open:

```
cd $HOME\vigil-mdr
.\.venv\Scripts\python.exe portal.py
```

Open http://127.0.0.1:8080 and sign in. From there you can review incidents, view process trees, and open HTML and PDF reports.

> **Check:** your incidents appear. Open one, select **Close**, then **Reopen**, and confirm both appear in its history.

Step 12: Publish the portal (optional)

> Publishing puts real hostnames, usernames, IP addresses and command lines behind one password. Skip this step unless you need it, and use a long, unique password. Keep the site private to people allowed to see that data.

Stop the portal with `Ctrl+C`, then start it in public mode:

```
$env:VIGIL_PUBLIC = '1'
.\.venv\Scripts\python.exe portal.py
```

In a second window, start the tunnel:

```
cloudflared tunnel --no-autoupdate --url http://127.0.0.1:8080
```

It prints an address ending in `trycloudflare.com`. Open it, and sign in there, because the login won't work on `127.0.0.1` in public mode. To take the site offline, press `Ctrl+C` in the tunnel window.

Your laptop stays the server, so the site stops if it sleeps or shuts down. The address changes whenever the tunnel restarts. A stable address needs a domain you own and a named Cloudflare tunnel.

Troubleshooting

Problem	What to do
Scripts are disabled	Run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`
`KeyError` for a variable	Open a new PowerShell window, then repeat the step that sets it
401 or 403 from LimaCharlie	Check the Organization ID, API key and `insight.det.get` permission
Empty detection results	Check Insight is enabled, and that rules fired in the last 3 days
Connection error to port 11434	Start Ollama
Connection error to port 5432	Start PostgreSQL
Wrong portal password	Run `tools\set-portal-password.ps1` again, then restart the portal
Too many login attempts	Wait 15 minutes, or restart the portal
Incident shows `failed`	The AI answer failed validation. Vigil keeps the incident for manual review
Analysis is very slow	Close other heavy programs. Each incident can take several minutes

Keep it safe

- Never commit `data/`, `.venv/`, API keys or passwords.
- If an API key leaks, delete it in LimaCharlie and create a new one.
- Check reports before sharing them, because masking only covers common argument names.
- Keep the test machine isolated, and restore a snapshot after attack tests.

Where to go next

- `instructions.md` covers daily use and settings.
- `docs/Vigil-MDR-Implementation.pdf` explains the design.
- `SPRINT_STATUS.md` lists remaining work.